

Enterprise DevSecOps Security Lab

Advanced Technical Walkthrough for DevSecOps, Cloud Security, AppSec, Kubernetes Security, and Detection Engineering Interviews

Prepared as a professional portfolio walkthrough for the Enterprise-DevSecOps GitHub project

Document Type	Technical Walkthrough
Project	Enterprise DevSecOps Security Lab
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions
Primary Goal	Demonstrate full secure software delivery lifecycle with evidence, remediation, and CI/CD enforcement
Audience	Hiring managers, DevSecOps engineers, AppSec engineers, cloud security reviewers, SOC/detection engineers

Executive positioning

This is not only a Kubernetes demo. It is an end-to-end DevSecOps security engineering case study that starts with vulnerable application behavior, moves through CI/CD scanning and container hardening, enforces Kubernetes controls, validates supply-chain integrity, detects runtime attacks, visualizes security telemetry, and documents risk treatment in a way that a professional security team can review.

Table of Contents

- 1. Executive Walkthrough
- 2. Project Architecture and Trust Boundaries
- 3. Application Security: Vulnerable App to Hardened App
- 4. Docker Hardening and Image Validation
- 5. Kubernetes Security Controls
- 6. GitOps and Drift Control
- 7. Supply Chain Security
- 8. Runtime Security and Attack Simulation
- 9. Monitoring and Security Observability
- 10. CI/CD Pipeline and Real Blocking Gates
- 11. Evidence, Reports, and Auditability
- 12. Risk Register and Remediation Strategy
- 13. Interview-Level Talking Points
- 14. Appendices: Commands and File Map

1. Executive Walkthrough

The Enterprise DevSecOps Security Lab demonstrates the complete lifecycle of modern cloud-native security engineering. The project begins with a deliberately vulnerable Flask application and then follows a structured path through static analysis, dependency scanning, Docker image hardening, Kubernetes deployment, GitOps control, supply chain integrity, runtime detection, monitoring, and final risk documentation.

A technical reviewer should be able to open the repository and understand three things quickly: what was attacked, what control detected or prevented the issue, and what evidence proves that the remediation worked. This walkthrough is written to support that review process. It explains the intent behind the design, not only the commands that were executed.

Portfolio value

The project is strongest when it is presented as a security assessment package: vulnerable behavior, exploit proof, secure remediation, retest proof, scanner output, risk decisions, and GitHub PR workflow. This is the difference between a lab folder and a professional DevSecOps portfolio.

1.1 Core Project Story

The story to present in interviews is simple: I built a vulnerable application, proved the weaknesses, secured the application and runtime, deployed it into Kubernetes, enforced security policies, added supply-chain controls, simulated runtime attacks, detected them with Falco, exposed the detections through Prometheus/Grafana, and then created professional evidence and reports.

- The vulnerable app remains in the repository to prove the offensive test cases and scanner detections.
- The hardened app runs separately through Dockerfile.hardened and uses app_hardened:app as the runtime entry point.
- The CI/CD workflows are separated into report-only workflows and blocking security gates.
- The Kubernetes layer demonstrates least privilege, pod hardening, NetworkPolicy, Pod Security, Gatekeeper admission control, and GitOps self-healing.
- The runtime layer demonstrates Falco detections, Falcosidekick metrics, Prometheus scraping, Grafana dashboards, and alert rules.

1.2 High-Level Control Map

The project is organized around eight security control families. Instead of treating the tools as a checklist, the walkthrough explains the objective of each control and the evidence that proves it worked.

- **Application Security:** Bandit, pip-audit, and manual retesting identify insecure code and dependencies. Evidence includes scanner reports, exploit screenshots, and hardened app tests.
- **Container Security:** Dockerfile.hardened, non-root execution, read-only filesystem validation, dropped capabilities, and Trivy scans prove that the remediated app runs in a hardened image.
- **Kubernetes Security:** RBAC, NetworkPolicy, Pod Security, and Gatekeeper prevent insecure workloads, excessive privileges, and non-compliant deployments.
- **GitOps Governance:** ArgoCD detects and repairs configuration drift so the cluster returns to Git-defined desired state.

- **Supply Chain Integrity:** Cosign, Syft, GHCR, SBOMs, and provenance artifacts prove image integrity and component transparency.
- **Runtime Detection:** Falco and Falcosidekick detect suspicious container behavior, sensitive file access, and privileged workload activity.
- **Security Observability:** Prometheus and Grafana convert raw runtime detections into measurable security telemetry and dashboards.
- **CI/CD Governance:** GitHub Actions and branch rules block unsafe changes before merge and preserve an auditable pull request workflow.

2. Project Architecture and Trust Boundaries

The lab architecture is intentionally layered. Each layer introduces a security control and a corresponding evidence point. The design is not just a deployment diagram; it is a security engineering flow where every control has a reason, a validation method, and an artifact.



2.1 Trust Boundaries

A useful interview answer is to explain the trust boundaries. The developer workstation, GitHub Actions runner, container registry, Kubernetes API server, node runtime, and monitoring stack are separate trust zones. A compromise in one zone should not automatically compromise every other zone.

- Developer to repository: controlled by PR rules, branch protection, and required checks.
- Repository to CI runner: controlled by least-privilege workflow permissions and security gates.
- CI runner to registry: controlled by GHCR authentication and release workflows.
- Registry to Kubernetes: controlled by image signing, labels, and Gatekeeper policy.
- Workload to node: controlled by SecurityContext, dropped capabilities, read-only filesystem, and Pod Security.
- Runtime to detection layer: controlled by Falco event collection and Prometheus scraping.

2.2 Design Principle

The project should be presented as defense-in-depth. No single control is the whole security story. SAST finds insecure patterns before build, dependency scanning finds vulnerable packages, Docker hardening reduces runtime blast radius, Kubernetes policy prevents unsafe deployment, GitOps detects drift, supply-chain controls prove provenance, and runtime detection catches behavior that static scanning cannot predict.

3. Application Security: Vulnerable App to Hardened App

The application security phase is the foundation of the project. A deliberately vulnerable Flask app is used to demonstrate SQL injection, command injection, and unsafe rendering. The professional value comes from showing the full loop: exploit proof, root cause, secure code, retest, and documentation.

3.1 SQL Injection Finding

The vulnerable route accepts a user-controlled id value and builds a SQL query in an unsafe way. The hardened route rejects non-numeric identifiers and uses a parameterized query. This is important because an interviewer may ask how the vulnerability was actually fixed, not only how it was detected.

```
# Vulnerable pattern
query = f"SELECT id, username, role FROM users WHERE id = '{user_id}'"

# Hardened pattern
cursor.execute(
    "SELECT id, username, role FROM users WHERE id = ?",
    (user_id,)
)
```

Retest result: the payload containing SQL syntax is rejected with HTTP 400, and the response states that only numeric user IDs are allowed. This proves the application no longer allows the untrusted input to reach SQL syntax construction.

3.2 Command Injection Finding

The vulnerable route passes user-controlled input into a shell command. The hardened route validates the host value and uses subprocess with a list of arguments and shell=False. This prevents shell metacharacters such as semicolons from being interpreted as commands.

```
# Vulnerable pattern
subprocess.run(f"ping -c 1 {host}", shell=True, capture_output=True, text=True)

# Hardened pattern
subprocess.run(
    ["ping", "-c", "1", host],
    shell=False,
    capture_output=True,
    text=True,
    timeout=5
)
```

Retest result: the command injection payload is rejected with HTTP 400 and never executes. This is stronger than simply blocking one string because the route uses validation and avoids shell interpretation entirely.

3.3 Unsafe Rendering Finding

The vulnerable application demonstrates unsafe rendering behavior. The hardened application escapes user input before returning it in an HTML response. This proves that input is treated as data, not executable markup.

```
from markupsafe import escape

@app.route("/hello")
def hello():
    name = request.args.get("name", "guest")
    safe_name = escape(name)
    return Response(f"<h1>Hello {safe_name}</h1>", mimetype="text/html")
```

Retest result: a script tag is displayed as escaped text rather than being interpreted as active markup. Security headers such as X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Cache-Control, and Content-Security-Policy provide additional browser-side hardening.

3.4 Dependency Vulnerability Management

The secure application dependency file is separated from the vulnerable requirements. This separation is important because it shows the secure build does not accidentally inherit vulnerable packages used only for demonstration. The hardened dependency file uses Flask 3.1.3 and Gunicorn 23.0.0, and pip-audit reports no known vulnerabilities for the secure requirements at scan time.

```
src/app/requirements-secure.txt

Flask==3.1.3
gunicorn==23.0.0
```

4. Docker Hardening and Image Validation

The Docker hardening phase is one of the most important technical interview points. A common weakness in portfolio projects is claiming that an app was hardened while the Dockerfile still runs the vulnerable code path. This project fixes that by making Dockerfile.hardened copy requirements-secure.txt and start app_hardened:app.

```
COPY src/app/requirements-secure.txt ./requirements.txt
COPY src/app/app_hardened.py ./app_hardened.py
USER 10001:10001
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app_hardened:app"]
```

4.1 Runtime Hardening Controls

- The container runs as UID 10001 rather than root.
- The runtime command starts app_hardened:app, not the vulnerable app.py.

- The container is tested with a read-only filesystem.
- Linux capabilities are dropped during runtime validation.
- The healthcheck confirms the service name hardened-flask-app.
- The Dockerfile configuration scan reports zero misconfigurations.

4.2 Why Base Image Findings Still Matter

Trivy can still report HIGH or CRITICAL vulnerabilities inherited from the Debian-based Python image. The professional response is not to hide those findings. The correct response is to distinguish application dependency risk from base-image operating system package risk and document a remediation or risk-treatment plan.

Important risk message

The hardened app is remediated, but the base image may still contain OS-level CVEs. A real enterprise workflow tracks these as base-image risk, tests patched or alternative base images, and uses ignore rules only with documented justification and expiry.

5. Kubernetes Security Controls

The Kubernetes layer demonstrates how a containerized application is deployed with workload-level controls. The strongest part of the project is that it does not stop at deployment; it validates security controls with attack-style tests and evidence screenshots.

5.1 SecurityContext and Pod Hardening

The hardened deployment should enforce a security context that reduces container breakout risk and limits runtime impact. The main controls include runAsNonRoot, readOnlyRootFilesystem, allowPrivilegeEscalation false, dropped capabilities, seccomp RuntimeDefault, and resource requests/limits.

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop:
      - ALL
  seccompProfile:
    type: RuntimeDefault
```

5.2 RBAC and Least Privilege

The RBAC phase intentionally creates an overprivileged ServiceAccount to demonstrate the risk of cluster-admin bindings. The remediation removes the dangerous ClusterRoleBinding and replaces it with a namespace-scoped RoleBinding that allows only the required action. This is a strong cloud security interview story because it shows both attack and remediation.

- Attack proof: the overprivileged ServiceAccount can access secrets and perform cluster-level actions.

- Audit proof: the dangerous ClusterRoleBinding is identified.
- Remediation proof: the ClusterRoleBinding is deleted and a restricted RoleBinding is created.
- Retest proof: the ServiceAccount can read ConfigMaps but cannot read secrets or create ClusterRoleBindings.

5.3 NetworkPolicy

NetworkPolicy demonstrates a default-deny approach. The purpose is to show that workload-to-workload and workload-to-internet communication is intentional rather than open by default. For a portfolio repo, it is important to include both the manifest and the validation evidence.

6. GitOps and Drift Control

GitOps is demonstrated with ArgoCD. The key story is not simply installing ArgoCD; the key story is proving desired state, detecting drift, and self-healing. A manual kubectl change creates drift, and ArgoCD reconciles the cluster back to the Git-defined state.

```
syncPolicy:  
  automated:  
    prune: true  
    selfHeal: true
```

This gives the project an enterprise operations angle: Kubernetes state should not depend on manual shell commands. Production-grade systems need auditable Git-driven deployment and automated reconciliation.

7. Supply Chain Security

The supply chain phase proves image integrity and transparency. The project includes image publishing, signing, SBOM generation, and provenance-style evidence. This is important for MNC-level DevSecOps roles because modern software delivery requires evidence about what was built, where it came from, and whether it was tampered with.

7.1 Image Signing and Verification

Cosign signing shows that the image digest can be associated with a cryptographic signature. The signature verification proof should be linked from the evidence index and described in the supply-chain report.

7.2 SBOM Generation

Syft-generated SBOMs provide a list of software components inside the image. The project should keep both SPDX and CycloneDX outputs because different enterprise teams may request different formats. The SBOM should not just exist; it should be summarized in a human-readable document that lists package count, top dependencies, and known vulnerability status.

7.3 Admission Policy for Supply Chain Metadata

OPA Gatekeeper is used to enforce required labels such as security-owner, sbom, and signed. The value of this control is traceability. A Kubernetes deployment should carry metadata proving ownership, SBOM status, and signing status.

8. Runtime Security and Attack Simulation

Runtime security is demonstrated through Falco. The project simulates behaviors such as sensitive file access, privileged pod activity, and hostPath abuse. Falco detections prove that the runtime layer can see suspicious activity after deployment.

8.1 Privileged Pod and HostPath Simulation

The privileged pod simulation demonstrates a common Kubernetes risk: a container with privileged mode and hostPath access can interact with the node filesystem. The remediation uses Pod Security restricted enforcement to prevent this class of workload from being admitted.

8.2 Detection Engineering View

Each Falco detection should be documented like a detection engineering rule: data source, trigger condition, severity, false positives, triage steps, response steps, and evidence screenshot. This makes the project valuable not only for DevSecOps roles but also for SOC and detection engineering roles.

9. Monitoring and Security Observability

The monitoring phase integrates Falco, Falcosidekick, Prometheus, and Grafana. This turns runtime security from log-only evidence into measurable telemetry. The custom Grafana dashboard shows detections, priority distribution, top triggered rules, kernel event processing rate, and drops.

9.1 Prometheus Targets and Metrics

Prometheus scrapes Falco and Falcosidekick metrics through ServiceMonitors. Target health screenshots prove that the metrics pipeline is active. Query screenshots prove that Falco rule matches can be measured over a time window.

9.2 Grafana Dashboard

The custom Falco dashboard is a strong portfolio asset. It shows that the project can move from raw events to operational visibility. The dashboard ConfigMap evidence also proves that dashboard-as-code is used rather than manual-only configuration.

10. CI/CD Pipeline and Real Blocking Gates

A major MNC-level improvement is the separation between report-only workflows and real blocking gates. A report-only workflow may use tolerant behavior to collect evidence. A blocking gate must fail the pull request when a critical policy is violated.

10.1 Workflow Separation

The project now uses a professional workflow structure. The CI workflow builds, tests, and uploads report artifacts. Security-gates.yml runs Bandit, pip-audit, Trivy config scanning, and workflow-quality checks. Additional workflows run CodeQL, Gitleaks, Trivy image scanning, ZAP baseline scanning, and supply-chain release automation.

```
.github/workflows/
├── ci.yml
├── ci-report-only.yml
├── security-gates.yml
├── codeql.yml
├── gitleaks.yml
├── trivy-container.yml
├── zap-baseline.yml
└── release-supply-chain.yml
```

10.2 No Fake Gates

The blocking workflows are checked to ensure they do not use patterns such as `|| true` or fake exit-code 0 behavior in the gate steps. This matters because a recruiter or technical interviewer may inspect the workflow and ask whether the gate actually blocks unsafe builds.

10.3 Branch Protection

The main branch ruleset requires pull requests and required status checks. In solo portfolio mode, the approval count can remain zero, but required checks must be enabled so that security automation has real enforcement. The required checks include build/test, Python SAST/dependency gates, config gates, workflow quality, CodeQL, and Gitleaks.

11. Evidence, Reports, and Auditability

A professional security project needs evidence that is easy to review. Screenshots alone are not enough; each screenshot should be mapped to a control, test, or finding. Scanner outputs should be saved under security/reports, and human-readable summaries should explain the result and remediation decision.

11.1 Recommended Evidence Index Format

Evidence ID	Area	What It Proves	File/Location
APP-001	SQL Injection	Vulnerable route exploited and later blocked	evidence/screenshots + APP finding
APP-002	Command Injection	Shell injection path blocked in hardened app	retest screenshot + secure code
DOCKER-001	Hardened Image	Dockerfile.hardened runs app_hardened:app	174-181 screenshots
K8S-001	Privileged Pod	HostPath risk detected and blocked	runtime/RBAC screenshots
CI-001	Security Gates	Blocking workflows have	182-192 screenshots

12. Risk Register and Remediation Strategy

The strongest professional version of this project should include a risk register. A risk register shows that the candidate can prioritize, track, and communicate security work. It also prevents scanner findings from looking ignored.

12.1 Example Risk Decisions

- Application dependency vulnerability in Flask: fixed by upgrading secure requirements to Flask 3.1.3.
- Dockerfile misconfiguration risk: remediated; Trivy Dockerfile scan reports zero misconfigurations.
- Base image OS CVEs: documented as base-image risk requiring review, patch testing, or justified ignore with expiry.
- Trivy container gate instability: not required in branch protection until base-image risk is treated.
- ZAP baseline noise: allowed to run but not yet required until thresholds and baseline policy are stable.

13. Interview-Level Talking Points

When asked to explain the project, avoid saying only that many tools were used. Focus on the lifecycle and the reason each tool exists.

30-second answer

I built a full DevSecOps security lab that starts with a vulnerable Flask app, proves SQL injection and command injection, remediates them in a hardened app, builds a non-root Docker image, deploys to Kubernetes with least privilege and policy controls, signs and documents the image supply chain, detects runtime attacks with Falco, visualizes security events in Grafana, and enforces real CI/CD security gates through GitHub branch protection.

13.1 Questions You Should Be Ready For

- How do you prove Dockerfile.hardened runs the remediated app?
- Why do you keep the vulnerable app in the repository?
- What is the difference between report-only CI and blocking gates?
- Why did Trivy still show HIGH/CRITICAL findings after hardening?
- How does RBAC least privilege prevent the simulated attack?
- How does ArgoCD detect and correct drift?
- What would you do before productionizing this lab?
- Which checks should be required in branch protection and why?

14. Appendices: Commands and File Map

14.1 Key Validation Commands

```
# Prove hardened image command
docker inspect devsecops-app:hardened --format='{{json .Config.Cmd}}'
```

```
# Healthcheck proof
curl -s http://localhost:5001/health | jq

# Injection retest proof
curl -s -i "http://localhost:5001/user?id=1'%20OR%20'1'%3D'1"
curl -s -i "http://localhost:5001/ping?host=127.0.0.1%3Bwhoami"

# Container hardening proof
docker exec devsecops-hardened id
docker exec devsecops-hardened sh -c 'touch /app/should-not-write 2>&1 || true'
docker exec devsecops-hardened sh -c 'grep CapEff /proc/1/status'

# CI workflow fake gate proof
grep -R "|| true" .github/workflows/security-gates.yml .github/workflows/gitLeaks.yml
.github/workflows/trivy-container.yml .github/workflows/zap-baseline.yml
.github/workflows/codeql.yml || echo PASS
```

14.2 Professional File Map

```
README.md
SECURITY_FINDINGS_REPORT.md
Dockerfile.hardened
src/app/app_hardened.py
src/app/requirements-secure.txt
.github/workflows/security-gates.yml
.github/workflows/codeql.yml
.github/workflows/gitLeaks.yml
.github/workflows/trivy-container.yml
.github/workflows/zap-baseline.yml
security/reports/
security/exceptions/base-image-vulnerability-review.md
docs/remediation/hardened-image-validation.md
docs/ci-cd/security-gates.md
docs/ci-cd/workflow-separation.md
evidence/screenshots/
```

Closing Assessment

With the hardened Dockerfile fix, real CI/CD gate separation, required status checks, base-image vulnerability review, and professional evidence structure, this project becomes much stronger than a normal student lab. It demonstrates security engineering judgment: not only using tools, but validating behavior, documenting risk, and enforcing controls through GitHub workflow and branch governance.

The next professional upgrades should be a complete SECURITY_FINDINGS_REPORT.md, a retest report, a control validation matrix, a STRIDE threat model, and an evidence index that maps every screenshot to a finding or control.